

Lab 6: TinyML for Anomaly Detection

Lab Report

Section 1: Edge Impulse Results

Public Edge Impulse Project Link

<https://studio.edgeimpulse.com/public/996745/live>

Answers to Part 1 Questions

1. Raw Inputs

The input signals to the Edge Impulse application consist of three-dimensional accelerometer data captured by the IMU sensor installed on the Arduino Nano 33 BLE Sense board - accX, accY, and accZ. Time series data is recorded using a sampling rate of 100 Hz with a window length of 2000 milliseconds and window stride of 220 milliseconds.

2. NN Classifier Features

The NN Classifier includes spectral characteristics computed using the Spectral Analysis module, which converts the input time-series from the accelerometers into frequency-domain characteristics to describe the fan vibration. The total number of input features is 33, corresponding to 11 spectral characteristics per axis across the three axes of the accelerometers. Some examples of the actual feature names used to train the classifier include accX RMS, accX Peak 1 Height, accX Peak 1 Frequency, accY Spectral Power 2.0-5.0, and accZ Spectral Power 2.0-5.0.

3. NN Classifier Outputs

The NN Classifier produces 2 output classes: nominal and off. These represent the two operating states of the fan (running normally vs. powered off) respectively.

4. Anomaly Detector Features

The anomaly detector uses a K-means clustering model trained on a subset of the spectral features rather than all 33. The features were chosen based on feature importance scores computed during the Spectral Analysis stage, and in this lab three features were selected (the ones marked with stars on the Anomaly Detection page): accX Peak 1 Height, accY Spectral Power 2.0 – 5.0, and accZ Spectral Power 2.0 – 5.0. The K-means model produces a single anomaly score representing the distance from the current input to the nearest cluster center in this 3-dimensional feature space; larger distances indicate behavior that does not match either of the learned clusters.

Deployment Evidence

Screenshot of the Edge Impulse model running on the Arduino Nano 33 BLE Sense via edge-impulse-run-impulse:

```

Inferencing settings:
  Interval: 10.00 ms.
  Frame size: 600
  Sample length: 2000 ms.
  No. of classes: 2
Starting inferencing, press 'b' to break
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 611 us, anomaly 505 us, postprocessing 1 ms
#Classification predictions:
  nominal: 0.402344
  off: 0.597656
Anomaly prediction: 12.054755
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 593 us, anomaly 507 us, postprocessing 999 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 16.024694
Anomaly prediction: 16.025049
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 595 us, anomaly 6 ms, postprocessing 49 us
#Classification predictions:
  nominal: 0.156250
  off: 0.843750
Anomaly prediction: 1000.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 593 us, anomaly 6 ms, postprocessing 49 us
#Classification predictions:
  nominal: 0.003906
  off: 0.996094
Anomaly prediction: 1000.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 612 us, anomaly 6 ms, postprocessing 50 us
#Classification predictions:
  nominal: 0.046875
  off: 0.953125
Anomaly prediction: 1000.000000

```

Short Interpretation

The deployed Edge Impulse model sampled acceleration data from the Nano 33 BLE Sense and provided classification probabilities (nominal/off) as well as an anomaly score using the K-means algorithm. In this case, because the board was stationary on the desk, the classification was poorly distributed, with 44/56 in favor of nominal/off, since the board was not mounted on the fan. The SoftMax output is irrelevant in this case, since neither training class matches the input data. However, the anomaly score was quite high (16), and therefore, it indicated that the input is out of distribution.

The classifier output cannot be considered reliable in the above case since it would have been the case even when the input doesn't correspond to any of the training classes, in other words, even a SoftMax classifier will output probabilities for all classes. It is useful to look at the anomaly score as an additional piece of information, since it indicates how far away from the training data the input was.

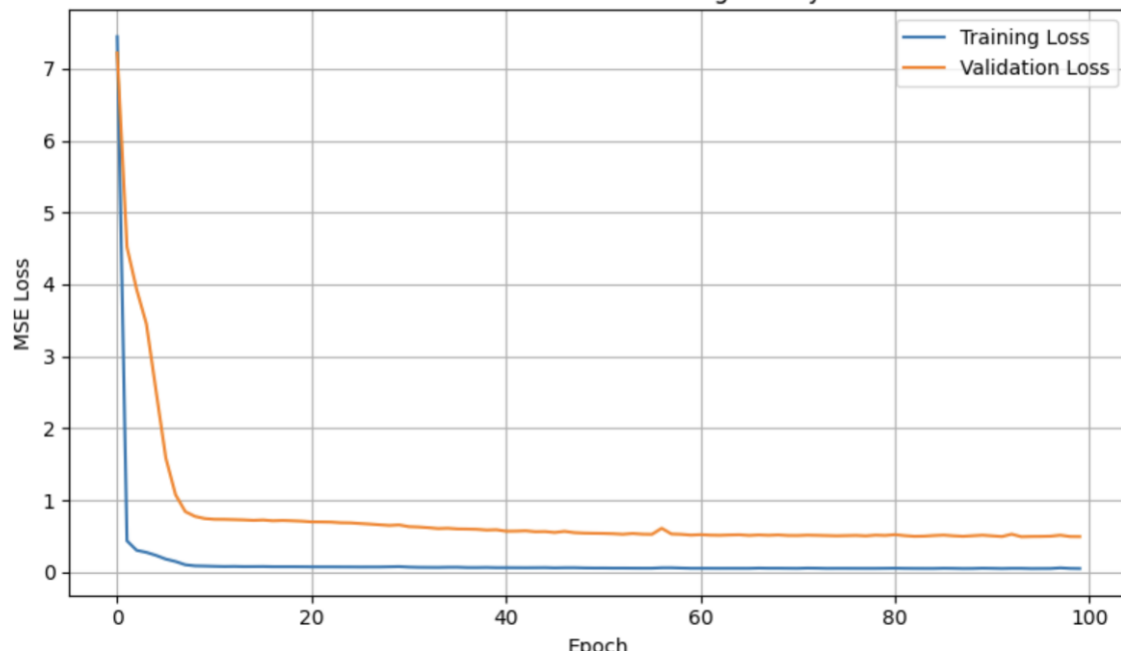
Section 2: Autoencoder Model Results

Selected Anomaly Threshold

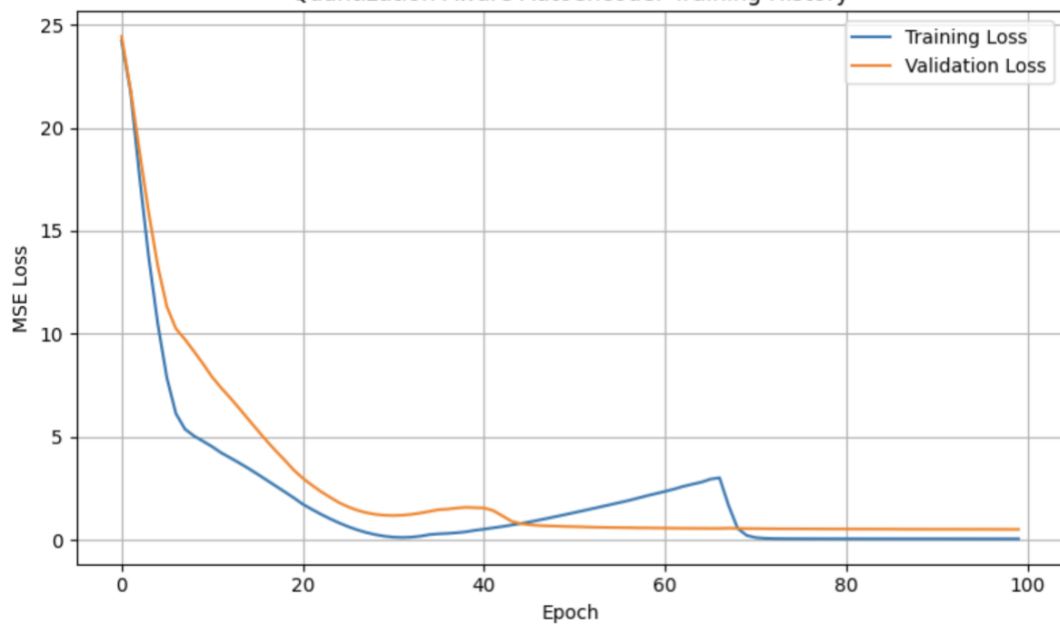
threshold = 0.5161743313074112

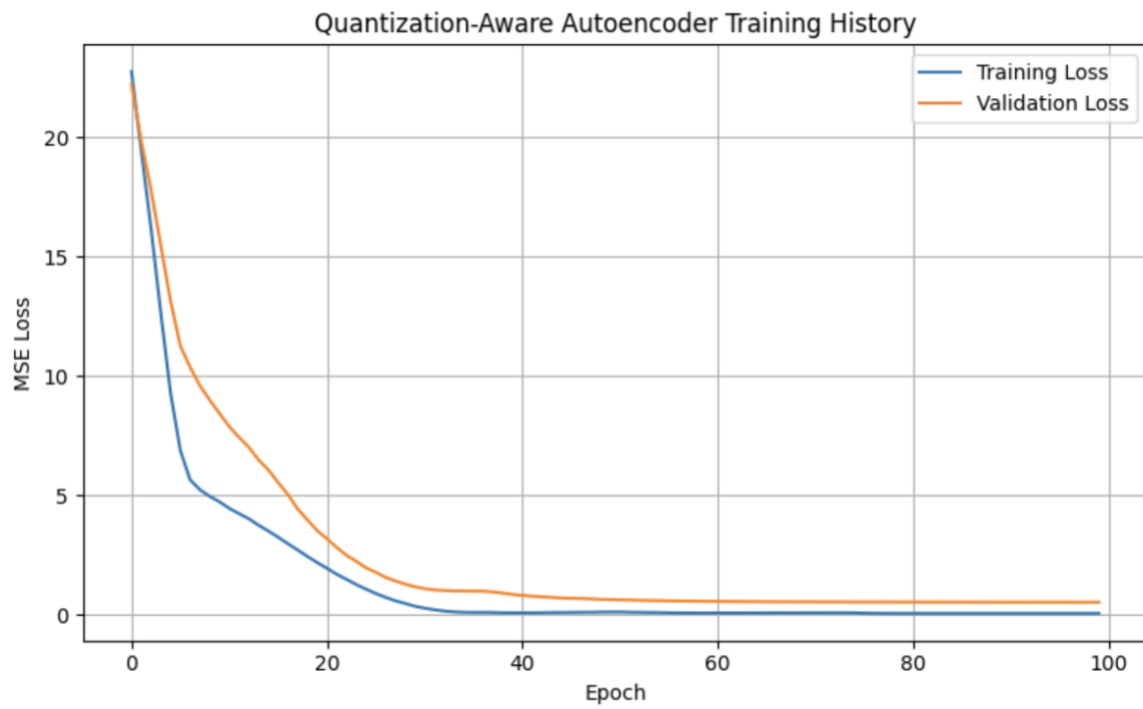
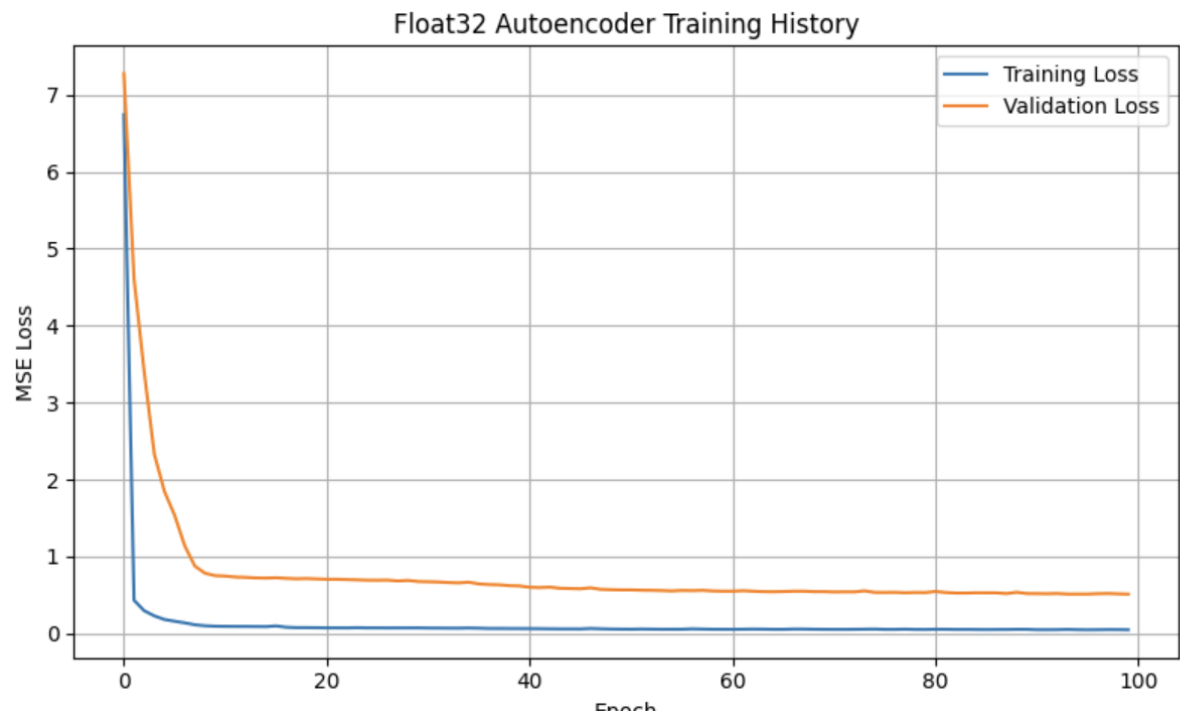
Training and Validation Loss Curves

Float32 Autoencoder Training History

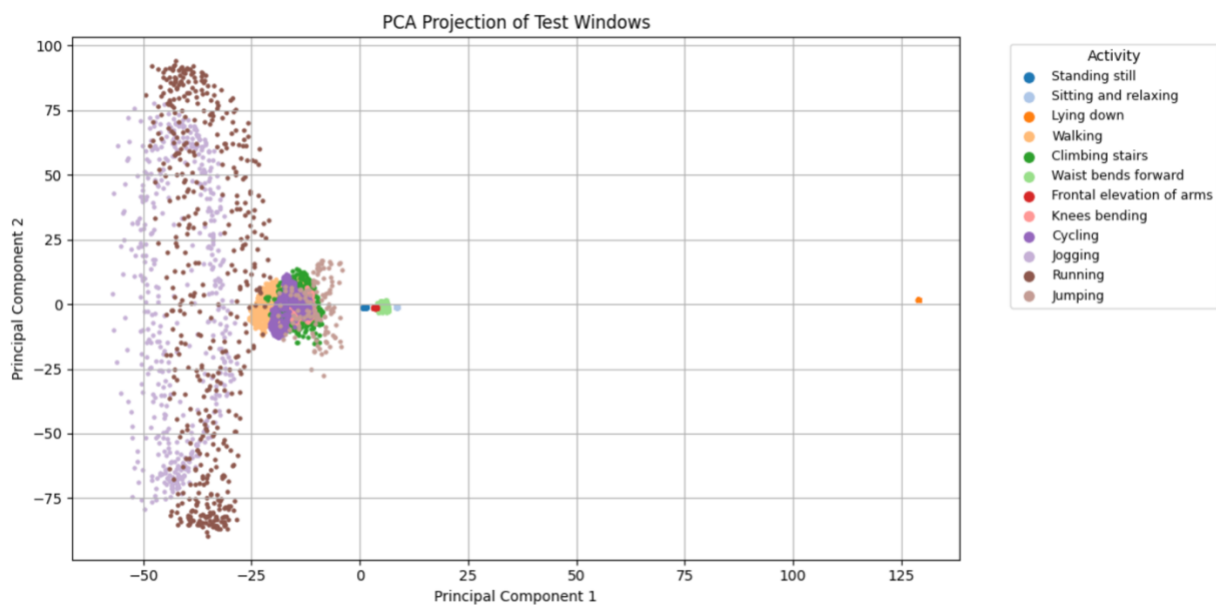
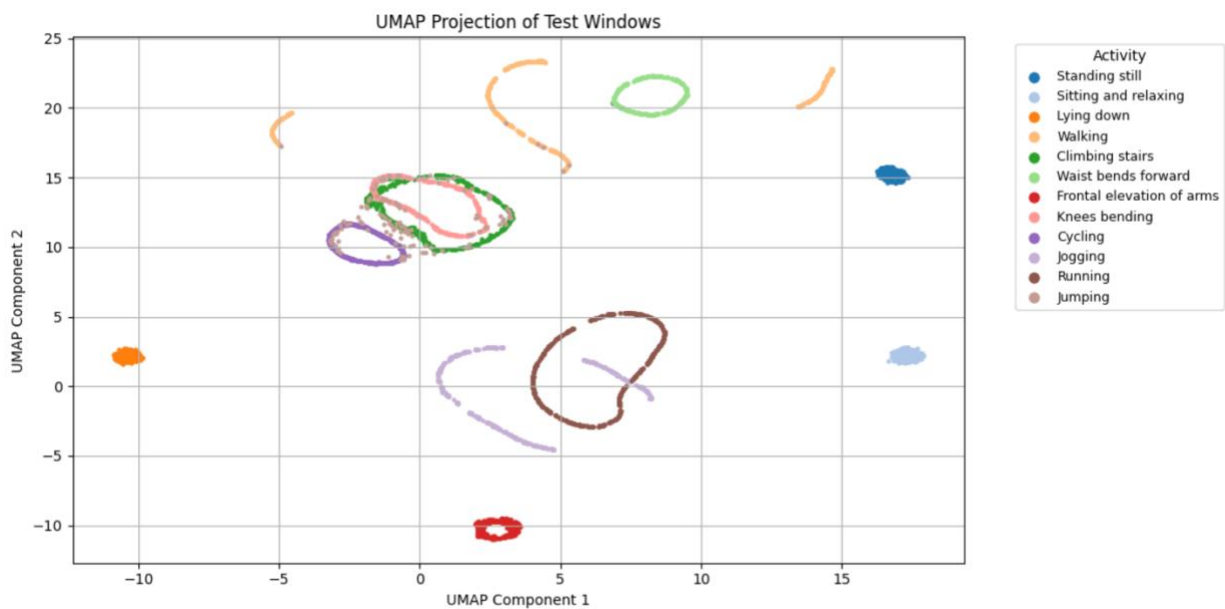


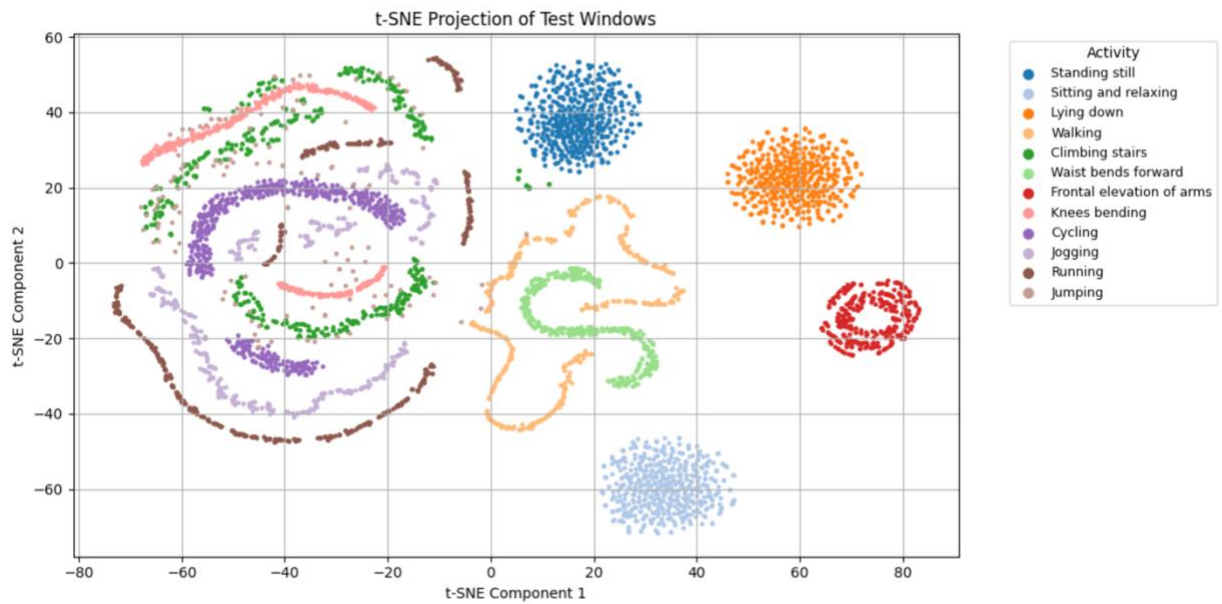
Quantization-Aware Autoencoder Training History





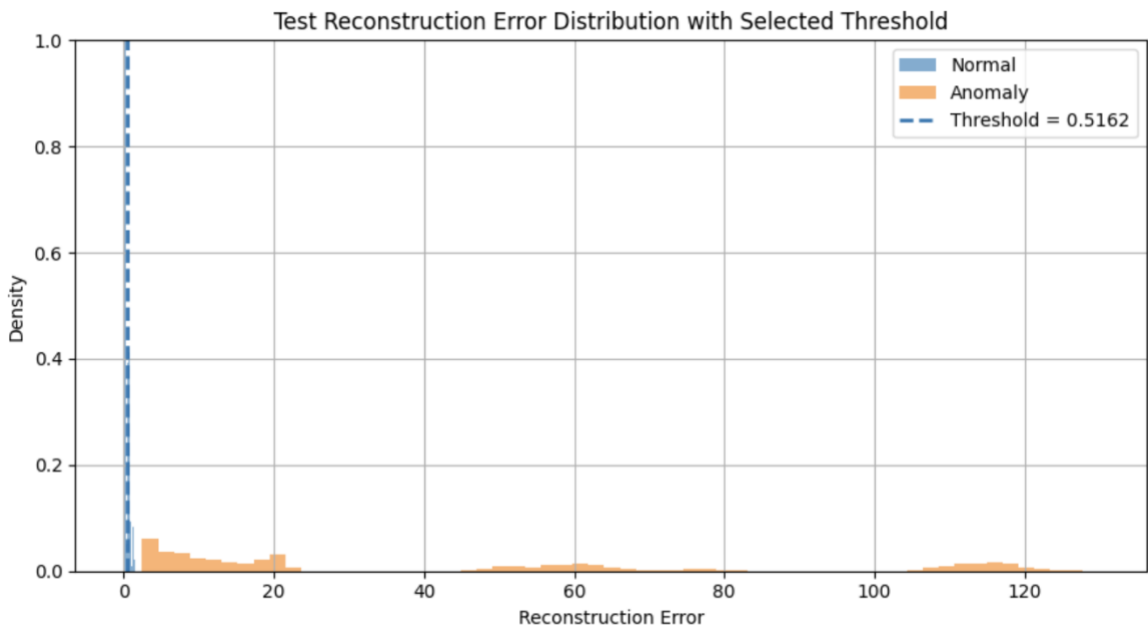
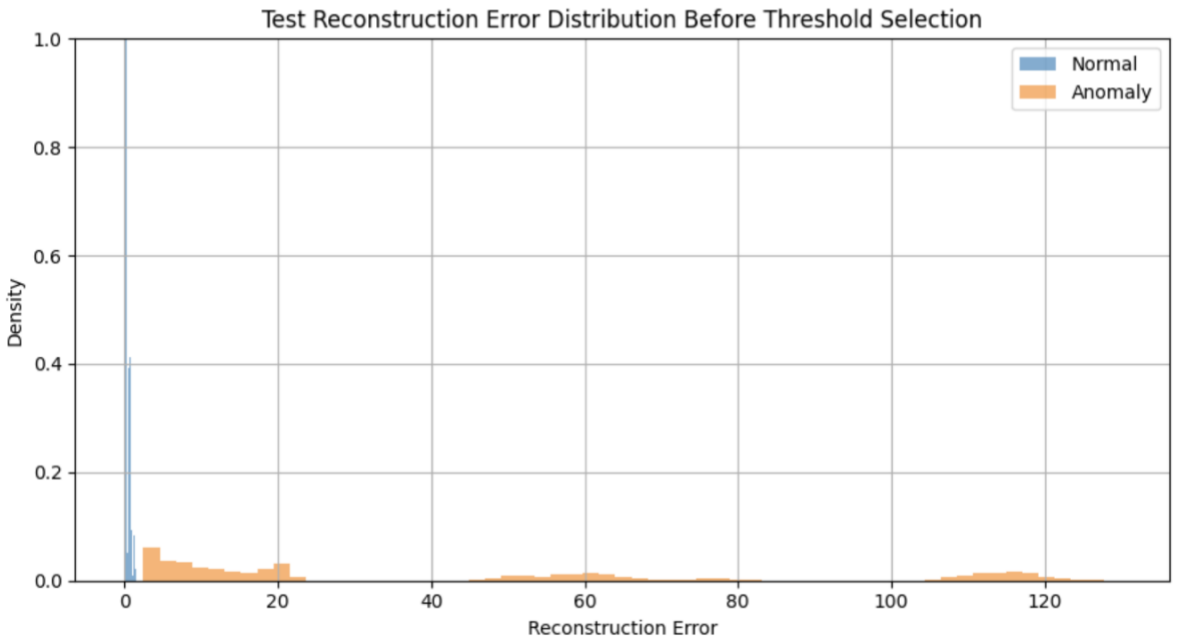
Dimensionality Reduction Visualizations





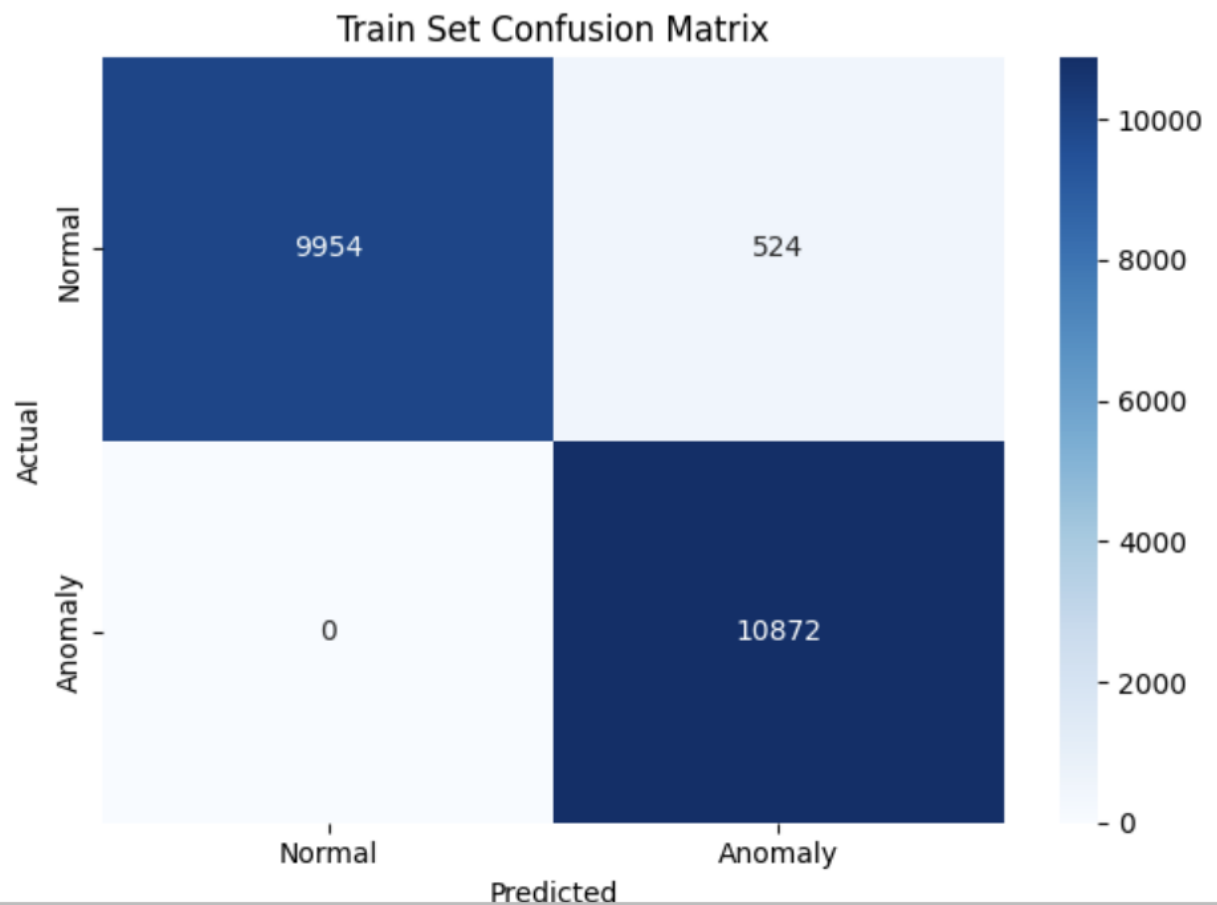
Reconstruction Error Distribution

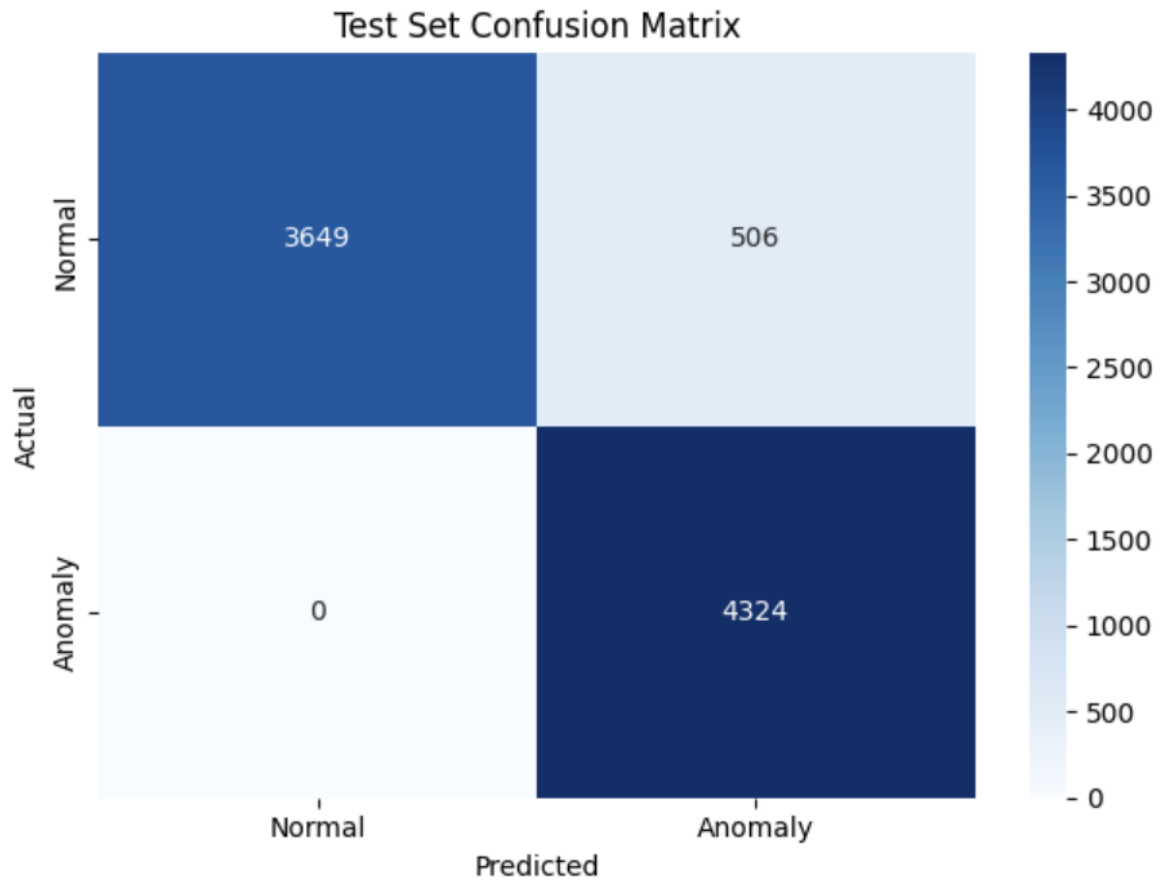
Train normal mean=0.102714, stderr=0.001656, min=0.009124, max=1.118055
Train anomaly mean=41.965240, stderr=0.373104, min=2.254362, max=128.425293
Test normal mean=0.139277, stderr=0.003877, min=0.009130, max=1.406886
Test anomaly mean=42.319565, stderr=0.624461, min=2.435190, max=129.792007



Confusion Matrix and Classification Report

Train set evaluation				
	precision	recall	f1-score	support
Normal (0)	1.00	0.95	0.97	10478
Anomaly (1)	0.95	1.00	0.98	10872
accuracy			0.98	21350
macro avg	0.98	0.97	0.98	21350
weighted avg	0.98	0.98	0.98	21350
Test set evaluation				
	precision	recall	f1-score	support
Normal (0)	1.00	0.88	0.94	4155
Anomaly (1)	0.90	1.00	0.94	4324
accuracy			0.94	8479
macro avg	0.95	0.94	0.94	8479
weighted avg	0.95	0.94	0.94	8479





Arduino Deployment Output

Serial Monitor output from the Arduino Nano 33 BLE Sense running the deployed int-8 quantized autoencoder:

```
Result: Normal activity
Reconstruction error: 0.147341
Result: Normal activity
Reconstruction error: 0.147338
Result: Normal activity
Reconstruction error: 0.147343
Result: Normal activity
Reconstruction error: 0.147347
Result: Normal activity
Reconstruction error: 0.147341
Result: Normal activity
Reconstruction error: 0.035247
Result: Normal activity
Reconstruction error: 0.035243
Result: Normal activity
Reconstruction error: 0.035242
Result: Normal activity
Reconstruction error: 0.035246
Result: Normal activity
Result: Anomaly detected
Reconstruction error: 2.609744
Result: Anomaly detected
Reconstruction error: 2.688116
Result: Anomaly detected
Reconstruction error: 2.771732
Result: Anomaly detected
Reconstruction error: 2.816664
Result: Anomaly detected
Reconstruction error: 2.886474
Result: Anomaly detected
Reconstruction error: 2.909013
```

Section 3: Discussion Questions

Q1: What threshold did you choose for anomaly detection, and why?

The selected threshold is 0.5162, calculated as the 95th percentile of normal training reconstruction errors. The idea is to use this threshold such that about 5 percent of normal training instances will have a reconstruction error exceeding it. It can be seen from the reconstruction error distribution that this point lies in the valley of both classes' distributions – the area where they intersect the least.

Q2: How does reconstruction error help distinguish normal and anomalous activity?

The autoencoder is trained solely using examples of normal movements (standing, sitting, lying down, waist bending, front arm lifting, and leg bends). As a result, it will learn how to compress and decompress normal movement data without making mistakes. If it receives data related to abnormal movements (walking, running, jogging, and jumping), then it will be outside the manifold of the normal movement data. The 16-dimensional layer cannot store enough data about the abnormal movements so that the output can be reconstructed accurately. This will cause a greater error rate when the model tries to reconstruct its input. This will give one scalar value that will distinguish abnormal movements from normal ones.

Q3: What information from the Python notebook must be preserved when deploying the model to Arduino?

Specific pieces of information need to be copied without modifications onto the device. Specifically, all bytes of the int-8-quantized model have to be saved into `autoencoder_model.cc` where they are stored as `g_model` byte array along with `g_model_len`. Additionally, input and output quantization values – namely, scale and zero point – must be taken into account, as these parameters can be obtained from the TFLite model at `input->params.scale`, `input->params.zero_point`, as well as respective values for outputs. The actual input shape and structure also have to be remembered as identical to that of the training data: 100 samples with 3 axes in the same order of X, Y, Z, flattened to a one-dimensional space of dimension 300, which will yield a binary decision of whether an observation is abnormal by using a threshold value of 0.5162. Furthermore, the accelerometer units must match those in the mHealth training data, which are accelerations measured in g on the Arduino.

Q4: Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

Reconstruction errors are expected to have a physical meaning when the samples presented to the algorithm during inference are identically distributed to those presented during training. The reconstruction errors for the autoencoder would be too high when the Arduino uses a different window size, stride, axis order, or when any of the normalizations used during training are skipped, because the data presented is out of the autoencoder's trained distribution and would thus yield erroneous reconstruction errors even for valid samples. The carefully calibrated threshold would thus fail to classify classes, resulting in either no readings or all readings being classified as anomalies. Preprocessing ensures that the trained threshold can be transferred from the computer to the microcontroller. For this experiment, the Arduino sketch

builds 100 sample windows using the same frequency and axis order as the model, and quantizes the floats into int-8s.

Q5: What are the main limitations of this anomaly detection method when deployed on a microcontroller?

There are several limitations that impact the applicability of this method on a microcontroller. Firstly, resource constraints, including memory and computational capacity, limit the model's size and depth and the window length. Namely, the small number of 16 bottleneck units limits the complexity of patterns the autoencoder can recognize, and the 64 KB tensor arena allows some, but not much, flexibility for architectural changes. Precision loss associated with Int-8 quantization may cause shifts in the distribution of errors generated by the model compared to the float32 results obtained in the Jupyter notebook, making it necessary to re-tune the threshold in practical use cases. The autoencoder will recognize activities seen during training, so any new patterns displayed by the user will be flagged as anomalies. Sensor drift, temperature changes, board orientation and mounting, and other factors that influence the data collection process but do not represent an anomaly can also change the input distribution. The IMU BMI270 sensor in the Arduino differs from the wearable ones used to collect the mHealth dataset. Furthermore, latency is not a problem for the small model, but power consumption is, since constant inference at 50 Hz requires significant energy and limits the duration of operation without charging. Finally, autoencoders cannot identify the type of activity; rather, they recognize that something is wrong, meaning that another step will need to be performed afterward.